

API Design Structures Explanation

Clean, minimal, scalable API blueprint used by good product companies, startups, and enterprise teams.

THE BEST & SIMPLEST API APPROACH (Universal for Any App)

- ✓ Scalable
- ✓ Fast
- ✓ Smooth
- ✓ Maintainable
- ✓ Easy to learn
- ✓ Easy to extend
- ✓ .NET 6 / 7 / 8 / 9 compatible

This is **Production-ready minimal architecture**

This structure works for

- Small APIs
- Large APIs
- Microservices
- Monolith APIs
- Internal tools
- Public APIs

WHY THIS STRUCTURE IS THE BEST

Requirement	Why This Works
Scalable	You add features without touching existing code
Fast	Minimal layers, minimal overhead
Maintainable	Clear responsibility per folder
Easy to learn	No over-engineering
Company standard	Matches real enterprise APIs
Future ready	Supports auth, caching, queues

API Design Structures Explanation

UNIVERSAL PROJECT STRUCTURE (Best Practice)

App.Api

```
|
|
|— Controllers          # HTTP layer only
|
|
|— Application
|
|   |— Interfaces      # Contracts
|   |— Services        # Business logic
|   |— DTOs            # Input/Output models
|
|
|— Domain
|   |— Entities        # Core models
|
|
|— Infrastructure
|   |— Persistence     # DbContext / Repositories
|   |— Configurations  # EF / External services
|
|
|— Common
|   |— Exceptions
|   |— Middleware
|   |— Helpers
|
|
|— Program.cs
|— appsettings.json
```

API Design Structures Explanation

STEP-BY-STEP: SIMPLEST API CREATION

STEP 1 Create Project

```
dotnet new webapi -n App.Api
```

```
cd App.Api
```

STEP 2 Domain Entity (Pure & Clean)

```
namespace App.Domain.Entities;
```

```
public class Item
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public bool IsActive { get; set; }
}
```

- ✓ No EF
 - ✓ No validation
 - ✓ No framework dependency
-

STEP 3 DTOs (API Safety Layer)

```
namespace App.Application.DTOs;
```

```
public record CreateItemDto(string Name);
public record ItemResponseDto(int Id, string Name, bool IsActive);
```

- ✓ Prevents exposing entity
 - ✓ Version-safe
-

STEP 4 Application Interface (Contract)

```
namespace App.Application.Interfaces;
```

```
public interface IItemService
```

API Design Structures Explanation

```
{  
    Task<ItemResponseDto> CreateAsync(CreateItemDto dto);  
    Task<IEnumerable<ItemResponseDto>> GetAllAsync();  
}
```

STEP 5 Application Service (Business Logic)

```
using App.Application.DTOs;  
using App.Application.Interfaces;  
using App.Domain.Entities;  
using App.Infrastructure.Persistence;  
  
namespace App.Application.Services;  
  
public class ItemService : IItemService  
{  
    private readonly AppDbContext _context;  
  
    public ItemService(AppDbContext context)  
    {  
        _context = context;  
    }  
  
    public async Task<ItemResponseDto> CreateAsync(CreateItemDto dto)  
    {  
        var item = new Item  
        {  
            Name = dto.Name,  
            IsActive = true  
        };  
  
        _context.Items.Add(item);  
    }  
}
```

API Design Structures Explanation

```
await _context.SaveChangesAsync();

return new ItemResponseDto(item.Id, item.Name, item.IsActive);
}

public async Task<IEnumerable<ItemResponseDto>> GetAllAsync()
{
    return _context.Items
        .Select(x => new ItemResponseDto(x.Id, x.Name, x.IsActive))
        .ToList();
}
}
```

- ✓ Business rules live here
 - ✓ Easy to test
 - ✓ No HTTP knowledge
-

STEP 6 Infrastructure (Persistence)

```
using Microsoft.EntityFrameworkCore;

using App.Domain.Entities;

namespace App.Infrastructure.Persistence;
```

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Item> Items => Set<Item>();
}
```

API Design Structures Explanation

STEP 7 Controller (Very Thin)

```
using Microsoft.AspNetCore.Mvc;
```

```
using App.Application.DTOs;
```

```
using App.Application.Interfaces;
```

```
namespace App.Api.Controllers;
```

```
[ApiController]
```

```
[Route("api/items")]
```

```
public class ItemsController : ControllerBase
```

```
{
```

```
    private readonly IItemService _service;
```

```
    public ItemsController(IItemService service)
```

```
    {
```

```
        _service = service;
```

```
    }
```

```
[HttpPost]
```

```
public async Task<ActionResult> Create(CreateItemDto dto)
```

```
{
```

```
    return Ok(await _service.CreateAsync(dto));
```

```
}
```

```
[HttpGet]
```

```
public async Task<ActionResult> GetAll()
```

```
{
```

```
    return Ok(await _service.GetAllAsync());
```

```
}
```

```
}
```

API Design Structures Explanation

- ✓ No logic
 - ✓ Easy to read
 - ✓ Easy to maintain
-

STEP 8 Program.cs (Minimal & Clean)

```
using App.Application.Interfaces;
using App.Application.Services;
using App.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Core
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// DB
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("AppDb"));

// DI
builder.Services.AddScoped<IItemService, ItemService>();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();
app.Run();
```

WHAT MAKES THIS API FAST & SMOOTH?

- ✓ Minimal layers
- ✓ No unnecessary abstractions
- ✓ DTO-based responses
- ✓ Clean DI
- ✓ Minimal middleware
- ✓ Easy caching addition
- ✓ Easy async scaling

How Teams Extend This (Without Breaking)

Feature	Where it goes
Validation	Application layer
Auth	Middleware
Caching	Application / Infrastructure
Logging	Middleware
Transactions	Service layer
Queues	Infrastructure

API Design Structures Explanation

FIRST: One Simple Mental Picture (Very Important)

Imagine a restaurant 🍽️

Real World	API
Customer	Client (Frontend / Mobile)
Waiter	Controller
Kitchen Manager	Service
Recipe Book	Domain
Storeroom	Database
Kitchen Tools	Infrastructure
Order Slip	DTO
Security Guard	Middleware

1 Controllers Folder

Controllers

What is it?

The entry gate of your API

Controllers **only receive request** and **send response**

What it should NOT do

-  Business logic
-  Database code
-  Validation rules

Restaurant Analogy

Waiter

- Takes order
- Passes it to kitchen
- Brings food back

Example

```
[ApiController]
```

```
[Route("api/items")]
```

API Design Structures Explanation

```
public class ItemsController : ControllerBase
{
    private readonly IItemService _service;

    public ItemsController(IItemService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<IActionResult> Create(CreateItemDto dto)
    {
        var result = await _service.CreateAsync(dto);
        return Ok(result);
    }
}
```

📌 Simple rule:

Controller = **HTTP only**

Application Folder

Application

What is it?

Brain of your application

This is where **business rules live**

Application → Interfaces

Why?

Defines **what your app can do**, not how.

Analogy

Menu card

API Design Structures Explanation

- Lists dishes
- Doesn't explain cooking

public interface IItemService

```
{  
    Task<ItemResponseDto> CreateAsync(CreateItemDto dto);  
}
```

Application → Services

Why?

Contains **actual business logic**

Analogy

Kitchen Manager

- Decides how dish is prepared
- Applies rules

public class ItemService : IItemService

```
{  
    public async Task<ItemResponseDto> CreateAsync(CreateItemDto dto)  
    {  
        // business rules  
    }  
}
```

Rule:

All decisions happen here

Application → DTOs

(You already understand this perfectly now 😊)

Why?

Safe data transfer between API and client.

```
public record CreateItemDto(string Name);
```

API Design Structures Explanation

Domain Folder

Domain

What is it?

Pure business data models

These represent **real things** in your system

Analogy

Recipe Book

- Knows what ingredients a dish has
 - Doesn't know who ordered it
-

Example

```
public class Item
```

```
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public bool IsActive { get; set; }  
}
```

Rules:

- ✓ No database code
 - ✓ No HTTP
 - ✓ No framework dependency
-

Infrastructure Folder

Infrastructure

What is it?

Technical implementation

How things are stored or fetched

Infrastructure → Persistence

Why?

API Design Structures Explanation

Handles **database communication**

Analogy

Storeroom

- Where ingredients are kept
-

Example

```
public class AppDbContext : DbContext
{
    public DbSet<Item> Items => Set<Item>();
}
```

Rule:

Database knowledge stays here

5 Common Folder

Common

What is it?

Reusable cross-cutting stuff

Used by many parts of the app

Common → Exceptions

Analogy

Complaint book

```
public class AppException : Exception
{
    public AppException(string message) : base(message) { }
}
```

Common → Middleware

Analogy

Security guard

API Design Structures Explanation

- Checks everyone before entry

```
public class ExceptionMiddleware
{
    public async Task Invoke(HttpContext context)
    {
        // global error handling
    }
}
```

Common → Helpers

Analogy

Utility tools

```
public static class DateHelper
{
    public static DateTime.UtcNow() => DateTime.UtcNow;
}
```

6 Program.cs

What is it?

Startup of the application

Wires everything together

Analogy

Restaurant opening routine

- Staff assigned
 - Doors opened
-

Example

```
builder.Services.AddScoped<IItemService, ItemService>();
app.MapControllers();
```

API Design Structures Explanation

7 appsettings.json

What is it?

Configuration file

No hard-coded values

Analogy

Restaurant settings

- Opening hours
- Location

```
{  
  "ConnectionStrings": {  
    "Default": "Server=..."  
  }  
}
```

ONE-LINE PURPOSE SUMMARY (Golden Table)

Folder	Purpose
Controllers	Handle HTTP
Application	Business rules
DTOs	Safe data transfer
Domain	Core models
Infrastructure	Technical work
Common	Shared logic
Program.cs	App startup

GOLDEN RULES (Remember Forever)

-  1 Controllers are **thin**
 -  2 Services are **smart**
 -  3 Entities are **pure**
 -  4 DTOs protect your API
-

API Design Structures Explanation

5 Infrastructure talks to DB

6 Common is shared stuff

Interview-Ready Explanation

“We separate responsibilities clearly: controllers handle HTTP, application layer handles business logic, domain contains core entities, infrastructure handles persistence, DTOs manage data transfer, and common handles cross-cutting concerns. This keeps the API clean, scalable, and maintainable.”

FULL REQUEST LIFECYCLE

👉 from the moment a client hits the API

👉 till data is saved in DB and response comes back

 FULL REQUEST LIFECYCLE (API → DB → API)

 First, the Big Picture (Memorize This)

Client

↓

Middleware

↓

Controller

↓

DTO

↓

Service (Business Logic)

↓

Domain Entity

↓

DbContext (Infrastructure)

↓

Database

↑

Response DTO

↑

Controller

↑

API Design Structures Explanation

Client

This is the **golden flow** used in real companies.

Real-World Analogy (Very Easy)

Imagine **ordering food in a restaurant** 🍔

Step	Real World	API World
1	Customer orders	Client sends HTTP request
2	Security checks	Middleware
3	Waiter takes order	Controller
4	Order slip	DTO
5	Kitchen prepares food	Service
6	Recipe	Domain Entity
7	Ingredients store	Database
8	Food served	Response

Keep this analogy in mind 🙌

🟢 STEP 1: Client Sends API Request

Client (Frontend / Postman)

POST /api/items

Content-Type: application/json

```
{  
  "name": "Laptop"  
}
```

🧠 Client only knows:

- URL
 - JSON
 - DTO format
-

🟢 STEP 2: Middleware Runs (Before Controller)

API Design Structures Explanation

What happens here?

- Logging
- Authentication
- Exception handling
- Validation (optional)

```
app.UseMiddleware<ExceptionMiddleware>();
```

🧠 Middleware is like **security gate**

- Runs for **every request**
 - Runs **before controller**
-

🟢 STEP 3: Controller Receives Request

[HttpPost]

```
public async Task<IActionResult> Create(CreateItemDto dto)
{
    return Ok(await _service.CreateAsync(dto));
}
```

Controller Responsibilities

- ✓ Accept request
- ✓ Convert JSON → DTO
- ✓ Call service
- ✓ Return response

✗ No logic

✗ No DB

🧠 Controller = **Traffic police**

🟢 STEP 4: DTO Is Created Automatically

DTO

```
public record CreateItemDto(string Name);
```

ASP.NET automatically converts:

```
{ "name": "Laptop" }
```

➡ into

```
new CreateItemDto("Laptop");
```

API Design Structures Explanation

 DTO is **order slip**

- Only allowed fields
 - No extra data
-

STEP 5: Service Layer (Business Logic)

```
public async Task<ItemResponseDto> CreateAsync(CreateItemDto dto)
{
    var item = new Item
    {
        Name = dto.Name,
        IsActive = true
    };

    _context.Items.Add(item);
    await _context.SaveChangesAsync();

    return new ItemResponseDto(item.Id, item.Name, item.IsActive);
}
```

What happens here?

- ✓ Business rules
- ✓ Mapping DTO → Entity
- ✓ Calls database
- ✓ Mapping Entity → Response DTO

 This is **THE HEART** of the API

STEP 6: Domain Entity Created

```
public class Item
{
    public int Id { get; set; }
    public string Name { get; set; }
    public bool IsActive { get; set; }
}
```

API Design Structures Explanation

}

🧠 Entity represents:

- Real data
- Database row
- Internal structure

✗ Client never sees this directly

🟢 STEP 7: DbContext Handles Persistence

```
_context.Items.Add(item);
```

```
await _context.SaveChangesAsync();
```

What EF Core does internally:

- 1 Tracks entity
- 2 Generates SQL
- 3 Executes INSERT
- 4 Gets generated ID

🧠 DbContext = **Translator**

- C# → SQL
-

🟢 STEP 8: Database Stores Data

```
INSERT INTO Items (Name, IsActive)
```

```
VALUES ('Laptop', 1);
```

🧠 Database is **final storage**

- Secure
 - Persistent
-

🟢 STEP 9: Response DTO Created

```
return new ItemResponseDto(
```

```
    item.Id,
```

```
    item.Name,
```

```
    item.IsActive
```

```
);
```

API Design Structures Explanation

Response DTO

```
public record ItemResponseDto(  
    int Id,  
    string Name,  
    bool IsActive  
);
```

🧠 Why Response DTO?

- ✓ Hide internal fields
- ✓ Clean response
- ✓ API contract

🟢 STEP 10: Controller Sends Response

```
{  
    "id": 1,  
    "name": "Laptop",  
    "isActive": true  
}
```

Controller wraps it in:

200 OK

🟢 STEP 11: Middleware Runs Again (After)

- Logging response
- Exception handling (if error)
- Cleanup

🧠 Middleware is both **before & after**

🔴 ERROR FLOW (Important)

Example: Invalid data

```
if (string.IsNullOrEmpty(dto.Name))  
    throw new ApplicationException("Name is required");
```

Flow:

API Design Structures Explanation

Service throws exception

→ Middleware catches

→ Returns clean error JSON

```
{  
  "message": "Name is required"  
}
```

 Controller never crashes.

COMPLETE FLOW IN ONE LINE (Memorize)

Client → Middleware → Controller → DTO → Service → Entity → DbContext → Database →
Response DTO → Client

Why This Lifecycle Is Best

- ✓ Clear responsibility
 - ✓ Easy debugging
 - ✓ Easy testing
 - ✓ Secure
 - ✓ Scalable
 - ✓ Maintainable
-

Interview-Perfect Answer

“When a request comes in, middleware processes it, the controller accepts the request and maps it to a DTO, the service applies business logic and converts DTOs to entities, the infrastructure layer persists data, and the response is mapped back to a response DTO and returned to the client.”

API Design Structures Explanation

DTOs in the simplest possible way, step-by-step, with real-world examples, before/after code, and why companies ALWAYS use them.

What is a DTO? (Plain English)

DTO = Data Transfer Object

A **DTO** is a **simple object** used **only to transfer data** between **API ↔ Client** (Request & Response)

 DTOs are **NOT database models**

 DTOs are **NOT business logic classes**

Real-World Analogy (Very Simple)

Imagine a Bank Counter

Thing	Real World	API World
Bank Vault	Secure storage	Database / Entity
Bank Form	Limited fields	DTO
Teller	Processing rules	Service
Customer	App / Frontend	Client

 **Customer never sees the vault**

 **Customer fills only a form (DTO)**

What Happens WITHOUT DTOs (Bad Practice)

Database Entity

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; } //  sensitive
    public bool IsAdmin { get; set; } //  internal
    public DateTime CreatedAt { get; set; } //  system field
}
```

API Design Structures Explanation

Controller (❌ Wrong)

[HttpPost]

```
public IActionResult Create(User user)
{
    _context.Users.Add(user);
    _context.SaveChanges();
    return Ok(user);
}
```

❌ Problems

🚨 Client can send:

```
{
  "name": "John",
  "email": "a@b.com",
  "isAdmin": true
}
```

🚨 Client can receive:

```
{
  "passwordHash": "...",
  "isAdmin": true
}
```

🚨 Breaking DB when schema changes

🚨 Security risk

🚨 No control

✅ What DTO Fixes (Purpose)

DTOs act as a **security + control layer**.

🎯 DTO Purposes

- 1 **Security** – hide sensitive fields
- 2 **Control** – allow only required inputs
- 3 **Stability** – DB changes won't break API
- 4 **Performance** – send only needed data

API Design Structures Explanation

5 Versioning – change API safely

6 Validation – clean input checks

DTO Types (Important)

DTO Type	Purpose
----------	---------

Request DTO	Data coming into API
-------------	-----------------------------

Response DTO	Data going out of API
--------------	------------------------------

Internal DTO	Communication between layers
--------------	------------------------------

Step-by-Step Example (Simple)

Step **1** Database Entity (Internal)

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal CostPrice { get; set; } //  internal
    public decimal SellingPrice { get; set; }
    public bool IsDeleted { get; set; } //  internal
}
```

Step **2** Request DTO (Client → API)

```
public record CreateProductDto(
    string Name,
    decimal SellingPrice
);
```

✓ Client cannot send CostPrice

✓ Client cannot send IsDeleted

Step **3** Response DTO (API → Client)

```
public record ProductResponseDto(
```

API Design Structures Explanation

```
int Id,  
string Name,  
decimal SellingPrice  
);  
✓ No internal data exposed  
✓ Clean response
```

Step 4 Service Uses DTOs

```
public async Task<ProductResponseDto> CreateAsync(CreateProductDto dto)  
{  
    var product = new Product  
    {  
        Name = dto.Name,  
        SellingPrice = dto.SellingPrice,  
        CostPrice = dto.SellingPrice * 0.8m, // business logic  
        IsDeleted = false  
    };  
  
    _context.Products.Add(product);  
    await _context.SaveChangesAsync();  
  
    return new ProductResponseDto(  
        product.Id,  
        product.Name,  
        product.SellingPrice  
    );  
}
```

Step 5 Controller (Very Clean)

```
[HttpPost]  
public async Task<ActionResult> Create(CreateProductDto dto)
```

API Design Structures Explanation

```
{  
    return Ok(await _service.CreateAsync(dto));  
}
```

Key Rule (Remember This Forever)

✗ Entities NEVER go outside API

✓ DTOs ALWAYS talk to the outside world

How Companies Think About DTOs

Why companies must use DTOs:

- ✓ Security compliance
 - ✓ GDPR / PII protection
 - ✓ API versioning
 - ✓ Microservice contracts
 - ✓ Clean testing
 - ✓ Multiple clients (Web, Mobile)
-

Simple Mental Model

Client ↔ DTO ↔ Service ↔ Entity ↔ Database

Interview-Perfect Answer

“DTOs are used to control and secure data transfer between the API and clients. They prevent exposing internal entities, allow validation, reduce payload size, and make APIs stable and maintainable.”

When DTOs Feel Like Overkill?

✗ Small throw away apps

✗ Demos

✗ Internal scripts

✓ But for ANY real app → DTOs are mandatory

API Design Structures Explanation

VALIDATION LIFECYCLE

- 👉 where validation REALLY belongs
- 👉 what should be validated at which layer
- 👉 with simple rules + real examples

🧠 First: The Golden Rule of Validation

- ❌ Validation does **NOT** belong in one place
- ✅ Validation belongs at **different layers for different reasons**

Think like **airport security** ✈️

Multiple checks, each for a different purpose.

🔄 Validation Lifecycle (Big Picture)

Client



Controller (Model Binding)



DTO Validation ✅



Service Validation ✅



Domain Validation ✅



Database Constraints ❌ (last safety)

Each layer validates **only what it is responsible for**.

🏠 Real-World Analogy (Very Simple)

Real World	API
Online Form	DTO Validation
Manager Approval	Service Validation
Company Policy	Domain Validation
Police	Database constraints

API Design Structures Explanation

1 DTO Validation (FIRST LINE OF DEFENSE)

Purpose

Validate **input format & basic rules**

“Is data even acceptable?”

What belongs here?

- ✓ Required fields
 - ✓ String length
 - ✓ Email format
 - ✓ Range checks
 - ✓ Null checks
-

What does NOT belong?

-  Database checks
 -  Business decisions
 -  Cross-entity rules
-

Example: DTO

```
public class CreateItemDto
{
    public string Name { get; set; } = string.Empty;
}
```

Using Data Annotations (Simple)

```
public class CreateItemDto
{
    [Required]
    [MinLength(3)]
    public string Name { get; set; } = string.Empty;
}
```

 If invalid → **Controller never calls Service**

Using FluentValidation (Company Standard)

API Design Structures Explanation

```
public class CreateItemDtoValidator : AbstractValidator<CreateItemDto>
{
    public CreateItemDtoValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MinimumLength(3);
    }
}
```

- ✓ Clean
 - ✓ Testable
 - ✓ Professional
-

2 Controller Validation (VERY MINIMAL)

Purpose

Check if request is **structurally valid**

```
if (!ModelState.IsValid)
    return BadRequest(ModelState);
```

 In modern APIs, this is automatic.

3 Service Validation (MOST IMPORTANT)

Purpose

Validate **business rules**

“Is this operation allowed?”

What belongs here?

- ✓ Duplicate checks
 - ✓ Permission checks
 - ✓ State checks
 - ✓ Workflow rules
-

Example

API Design Structures Explanation

```
public async Task<ItemResponseDto> CreateAsync(CreateItemDto dto)
{
    var exists = await _context.Items.AnyAsync(x => x.Name == dto.Name);
    if (exists)
        throw new AppException("Item already exists");

    // business logic continues...
}
```

🧠 This validation **cannot** be done in DTO.

4 Domain Validation (Business Invariants)

🧠 Purpose

Protect **core business rules** that must never break

✅ Example

```
public class Item
{
    public string Name { get; private set; }

    public Item(string name)
    {
        if (string.IsNullOrEmpty(name))
            throw new DomainException("Item name cannot be empty");

        Name = name;
    }
}
```

🧠 Even if someone bypasses Service,
Domain still protects itself.

5 Database Validation (LAST SAFETY NET)

API Design Structures Explanation

Purpose

Data integrity, not business logic

Examples

✓ Unique index

✓ Not null

✓ Foreign key

```
modelBuilder.Entity<Item>()
```

```
.HasIndex(x => x.Name)
```

```
.IsUnique();
```

 DB validation is **not user-friendly**

 Never rely on it for messages.

Validation Responsibilities Table (Very Important)

Layer	What to Validate
-------	------------------

DTO	Input correctness
-----	-------------------

Controller	Model binding
------------	---------------

Service	Business rules
---------	----------------

Domain	Invariants
--------	------------

Database	Integrity only
----------	----------------

Common Validation Mistakes (Avoid These)

 Business logic in DTO

 DB checks in Controller

 Validation inside Controller methods

 Only DB validation

 Duplicate validation everywhere

Perfect Validation Flow (Example)

Request:

```
{ "name": "" }
```

Flow:

API Design Structures Explanation

- 1 DTO Validation fails
 - 2 Controller returns 400
 - 3 Service not called
 - 4 DB untouched
-

Another Request:

```
{ "name": "Laptop" }
```

Flow:

- 1 DTO passes
 - 2 Service checks duplicate
 - 3 Saves to DB
 - 4 Returns success
-

Interview-Perfect Answer

“Validation is layered. DTO validation ensures input correctness, service validation enforces business rules, domain validation protects invariants, and database constraints act as a last safety net. This keeps APIs clean and maintainable.”

API Design Structures Explanation

Authentication & Authorization lifecycle in the **same simple, step-by-step way**, with **real-world analogy**, **clear responsibility**, and **minimal code**.

Authentication vs Authorization (First – VERY CLEAR)

In one line

Term	Meaning
Authentication	Who are you?
Authorization	What are you allowed to do?

Real-World Analogy (Best Way to Remember)

Office Building

Step	Real World	API
1	Show ID card	Authentication
2	Security checks ID	Token validation
3	Access allowed to floors	Authorization
4	Enter room	API action executed

FULL AUTH LIFECYCLE (BIG PICTURE)

Client



Login API



Authentication (Verify identity)



JWT Token issued



Client stores token



Client sends token with every request



Middleware validates token



API Design Structures Explanation

Authorization checks permission

↓

Controller → Service → DB

This is the **standard lifecycle used in companies**.

1 AUTHENTICATION LIFECYCLE (Login Flow)

● Step 1: Client Sends Login Request

POST /api/auth/login

Content-Type: application/json

```
{  
  "email": "user@test.com",  
  "password": "123456"  
}
```

🧠 Client proves **identity credentials**.

● Step 2: Controller Receives Login Request

```
[HttpPost("login")]
```

```
public async Task<ActionResult> Login(LoginDto dto)
```

```
{  
    var token = await _authService.LoginAsync(dto);  
    return Ok(new { token });  
}
```

✓ No validation logic

✓ No token creation logic

● Step 3: Service Validates Credentials (Authentication)

```
public async Task<string> LoginAsync(LoginDto dto)
```

```
{  
    var user = await _context.Users  
        .FirstOrDefaultAsync(x => x.Email == dto.Email);
```

API Design Structures Explanation

```
if (user == null || !VerifyPassword(dto.Password, user.PasswordHash))  
    throw new AppException("Invalid credentials");  
  
return GenerateJwtToken(user);  
}
```

 This is **AUTHENTICATION**

“Are you really who you say you are?”

Step 4: JWT Token Generated

```
private string GenerateJwtToken(User user)  
{  
    var claims = new[]  
    {  
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),  
        new Claim(ClaimTypes.Role, user.Role)  
    };  
  
    // token creation logic  
}
```

 Token contains:

- User ID
- Role
- Expiry
- Signature

✓ No password inside token

✓ Token is signed (cannot be modified)

Step 5: Client Stores Token

Client stores token in:

- Memory
-

API Design Structures Explanation

- Secure storage
- HttpOnly cookie (recommended for web)

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

2 AUTHORIZATION LIFECYCLE (Every Protected API Call)

● Step 6: Client Calls Protected API

GET /api/items

Authorization: Bearer <TOKEN>

🧠 Client proves **identity using token**, not password.

● Step 7: Authentication Middleware Runs

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

Middleware does:

- ✓ Token exists?
- ✓ Token valid?
- ✓ Token expired?
- ✓ Signature valid?

✗ If failed → **401 Unauthorized**

● Step 8: Authorization Checks Permission

Role-based authorization

```
[Authorize(Roles = "Admin")]
```

```
[HttpPost]
```

```
public IActionResult Create()
```

```
{
```

```
    // only admin can create
```

```
}
```

🧠 This is **AUTHORIZATION**

“Are you allowed to do this?”

✗ If failed → **403 Forbidden**

API Design Structures Explanation

Step 9: Controller Executes Normally

If:

- ✓ Token valid
- ✓ Permission valid

Then:

Controller → Service → DB → Response

WHO DOES WHAT? (VERY IMPORTANT)

Layer	Responsibility
Controller	Declare required permission
Middleware	Validate token
Service	Business-level permission checks
Domain	Critical rules
Database	No auth logic

Common Mistakes (Avoid These)

- ✗ Authorization logic in controller
 - ✗ Token validation in service
 - ✗ Role checks everywhere
 - ✗ Password checks after login
 - ✗ Storing sensitive data in JWT
-

FULL FLOW WITH ERROR EXAMPLES

✗ No token

401 Unauthorized

✗ Token valid but role not allowed

403 Forbidden

✗ Token expired

401 Unauthorized

API Design Structures Explanation

AUTHENTICATION vs AUTHORIZATION (FINAL TABLE)

Feature	Authentication	Authorization
Question	Who are you?	What can you do?
Happens	Login	Every request
Uses	Password / Token	Roles / Claims
Handled by	Service + Middleware	Middleware + Attributes

Interview-Perfect Answer

“Authentication verifies user identity and issues a JWT token. The client sends this token with every request. Middleware validates the token, and authorization checks roles or claims before allowing access to protected resources.”

JWT setup step-by-step in the **cleanest, simplest, company-standard way**.

WHAT, WHY, and WHERE — with **minimal but complete code**.

JWT SETUP — STEP BY STEP (REAL API WAY)

First: What JWT Does (1-line)

JWT allows the client to **prove identity** on every request **without sending password again**.

BIG PICTURE FLOW (MEMORIZE THIS)

Login API



Validate user credentials



Generate JWT token



Return token to client



Client sends token in headers



JWT Middleware validates token



Authorization allows / blocks request

STEP 0 Install Required Packages

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

STEP 1 JWT Settings (appsettings.json)

 **appsettings.json**

```
{
  "Jwt": {
    "Key": "THIS_IS_A_SUPER_SECRET_KEY_12345",
    "Issuer": "App.Api",
    "Audience": "App.Client",
    "ExpiryMinutes": 60
  }
}
```

 Why?

- **Key** → used to sign token
 - **Issuer** → who issued token
 - **Audience** → who can use token
 - **Expiry** → auto logout
-

STEP 2 Configure JWT in Program.cs

This is where **authentication middleware is wired**.

 **Program.cs**

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
```

```
using Microsoft.IdentityModel.Tokens;
```

```
using System.Text;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Controllers & Swagger
```

```
builder.Services.AddControllers();
```


API Design Structures Explanation

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
// 🗝️ JWT Authentication Setup
```

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```

```
.AddJwtBearer(options =>
```

```
{
```

```
    options.TokenValidationParameters = new TokenValidationParameters
```

```
    {
```

```
        ValidateIssuer = true,
```

```
        ValidateAudience = true,
```

```
        ValidateLifetime = true,
```

```
        ValidateIssuerSigningKey = true,
```

```
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
```

```
        ValidAudience = builder.Configuration["Jwt:Audience"],
```

```
        IssuerSigningKey = new SymmetricSecurityKey(
```

```
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)
```

```
        )
```

```
    };
```

```
});
```

```
builder.Services.AddAuthorization();
```

```
var app = builder.Build();
```

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

```
// 🗝️ ORDER MATTERS
```

```
app.UseAuthentication(); // Who are you?
```

API Design Structures Explanation

```
app.UseAuthorization(); // What can you do?
```

```
app.MapControllers();
```

```
app.Run();
```

IMPORTANT

UseAuthentication() MUST come before UseAuthorization()

STEP Login DTO

LoginDto.cs

```
public record LoginDto(string Email, string Password);
```

STEP Auth Service (Generate Token)

This is where **JWT token is created**.

AuthService.cs

```
using Microsoft.IdentityModel.Tokens;
```

```
using System.IdentityModel.Tokens.Jwt;
```

```
using System.Security.Claims;
```

```
using System.Text;
```

```
public class AuthService
```

```
{
```

```
    private readonly IConfiguration _config;
```

```
    public AuthService(IConfiguration config)
```

```
    {
```

```
        _config = config;
```

```
    }
```

```
    public string GenerateToken(int userId, string role)
```

```
    {
```

```
        var claims = new[]
```

API Design Structures Explanation

```
{  
    new Claim(ClaimTypes.NameIdentifier, userId.ToString()),  
    new Claim(ClaimTypes.Role, role)  
};  
  
var key = new SymmetricSecurityKey(  
    Encoding.UTF8.GetBytes(_config["Jwt:Key"]!)  
);  
  
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);  
  
var token = new JwtSecurityToken(  
    issuer: _config["Jwt:Issuer"],  
    audience: _config["Jwt:Audience"],  
    claims: claims,  
    expires: DateTime.UtcNow.AddMinutes(  
        int.Parse(_config["Jwt:ExpiryMinutes"]!)  
    ),  
    signingCredentials: creds  
);  
  
return new JwtSecurityTokenHandler().WriteToken(token);  
}
```

 Token contains:

- User ID
- Role
- Expiry
- Signature

API Design Structures Explanation

STEP 5 Register AuthService (DI)

Program.cs

```
builder.Services.AddScoped<AuthService>();
```

STEP 6 Login Controller

AuthController.cs

```
[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly AuthService _authService;

    public AuthController(AuthService authService)
    {
        _authService = authService;
    }

    [HttpPost("login")]
    public IActionResult Login(LoginDto dto)
    {
        //  Normally validate user from DB
        if (dto.Email != "admin@test.com" || dto.Password != "123456")
            return Unauthorized("Invalid credentials");

        var token = _authService.GenerateToken(userId: 1, role: "Admin");

        return Ok(new { token });
    }
}
```

API Design Structures Explanation

STEP 7 Protect API with [Authorize]

 ItemsController.cs

```
[Authorize]

[HttpGet]

public IActionResult GetAll()
{
    return Ok("You are authenticated");
}
```

Role-based protection

```
[Authorize(Roles = "Admin")]

[HttpPost]

public IActionResult Create()
{
    return Ok("Admin access only");
}
```

STEP 8 Client Sends Token

HTTP Request

GET /api/items

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

 Password is **never sent again**.

FULL JWT LIFECYCLE (FINAL)

Login API

- Validate credentials
 - Generate JWT
 - Client stores token
 - Client sends token
 - JWT middleware validates
 - Authorization checks role
 - Controller executes
-

Common JWT Mistakes (Avoid These)

-  Putting sensitive data in JWT
 -  Storing JWT in localStorage (web apps)
 -  Missing token expiry
 -  Skipping HTTPS
 -  Business logic inside controller
-

Interview-Perfect Explanation

“JWT authentication issues a signed token after successful login. The client sends this token with every request. Middleware validates the token and authorization checks roles or claims before allowing access.”

Refresh Token flow is what separates **beginner APIs** from **production-grade APIs**.

slowly, clearly, and practically — same style as before.

REFRESH TOKEN FLOW (PRODUCTION STANDARD)

First: Why Refresh Token Exists (Very Simple)

Problem with only JWT

- JWT expires (good for security)
- User gets logged out frequently (bad UX)

Solution

Use **short-lived JWT + long-lived Refresh Token**

One-Line Definition

Token	Purpose	Lifetime
JWT (Access Token)	Access APIs	Short (5–15 min)
Refresh Token	Get new JWT	Long (7–30 days)

Real-World Analogy (BEST WAY)

Movie Theatre

Real World	API
Entry Ticket	JWT

API Design Structures Explanation

Real World	API
Membership Card	Refresh Token
Ticket Expired	JWT expired
Show membership	Refresh token
New ticket issued	New JWT
👉 You don't log in again	
👉 You prove long-term identity	

COMPLETE REFRESH TOKEN LIFECYCLE (BIG PICTURE)

Login



JWT + Refresh Token issued



Client uses JWT



JWT expires



Client sends Refresh Token



Server validates Refresh Token



New JWT issued



Client continues

This is **exactly how real apps work**.

WHERE REFRESH TOKEN IS STORED

Server (Database)

- RefreshToken
 - UserId
-

API Design Structures Explanation

- ExpiryDate
- IsRevoked

Client

- HttpOnly Cookie (Web)
- Secure storage (Mobile)

✗ Never in JavaScript localStorage (for web)

1 DATABASE MODEL (Refresh Token)

```
public class RefreshToken
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public string Token { get; set; } = string.Empty;
    public DateTime ExpiresAt { get; set; }
    public bool IsRevoked { get; set; }
}
```

Why DB?

- Can revoke tokens
 - Can logout users
 - Can track sessions
-

2 LOGIN FLOW (JWT + REFRESH TOKEN)

AuthService (Login)

```
public async Task<AuthResponseDto> LoginAsync(User user)
{
    var jwt = GenerateJwtToken(user.Id, user.Role);

    var refreshToken = new RefreshToken
    {
        UserId = user.Id,
        Token = Guid.NewGuid().ToString(),
    }
```


API Design Structures Explanation

```
ExpiresAt = DateTime.UtcNow.AddDays(7)

};

_context.RefreshTokens.Add(refreshToken);
await _context.SaveChangesAsync();

return new AuthResponseDto(jwt, refreshToken.Token);
}
```

Response DTO

```
public record AuthResponseDto(string AccessToken, string RefreshToken);
```

3 CLIENT STORES TOKENS

Web App (Best Practice)

- JWT → Memory
- Refresh Token → **HttpOnly Cookie**

Mobile App

- Secure storage
-

4 ACCESS TOKEN EXPIRES (NORMAL)

Client calls API → gets:

401 Unauthorized

 This is expected.

Now **refresh flow starts**.

5 REFRESH TOKEN API

Request

POST /api/auth/refresh

Cookie: refreshToken=abc123

Controller

API Design Structures Explanation

```
[HttpPost("refresh")]
```

```
public async Task<ActionResult> Refresh([FromBody] RefreshRequestDto dto)
{
    var tokens = await _authService.RefreshAsync(dto.RefreshToken);
    return Ok(tokens);
}
```

DTO

```
public record RefreshRequestDto(string RefreshToken);
```

6 REFRESH TOKEN VALIDATION (Service)

```
public async Task<AuthResponseDto> RefreshAsync(string refreshToken)
{
    var token = await _context.RefreshTokens
        .FirstOrDefaultAsync(x =>
            x.Token == refreshToken &&
            !x.IsRevoked &&
            x.ExpiresAt > DateTime.UtcNow);

    if (token == null)
        throw new AppException("Invalid refresh token");

    // Rotate refresh token (security)
    token.IsRevoked = true;

    var newRefreshToken = new RefreshToken
    {
        UserId = token.UserId,
        Token = Guid.NewGuid().ToString(),
        ExpiresAt = DateTime.UtcNow.AddDays(7)
    };
}
```

API Design Structures Explanation

```
_context.RefreshTokens.Add(newRefreshToken);  
await _context.SaveChangesAsync();  
  
var newJwt = GenerateJwtToken(token.UserId, "User");  
  
return new AuthResponseDto(newJwt, newRefreshToken.Token);  
}
```

🧠 This is called **Refresh Token Rotation**

✓ Prevents token theft reuse

7 CLIENT UPDATES TOKENS

- Replace old JWT
- Replace refresh token
- Continue API calls

User **never notices** anything 🌀

8 LOGOUT FLOW (IMPORTANT)

Logout API

[Authorize]

[HttpPost("logout")]

```
public async Task<IActionResult> Logout()  
{  
    var userId = int.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier)!);  
  
    var tokens = _context.RefreshTokens  
        .Where(x => x.UserId == userId);  
  
    foreach (var token in tokens)  
        token.IsRevoked = true;  
}
```

API Design Structures Explanation

```
await _context.SaveChangesAsync();  
  
return Ok();  
}
```

 Logout = **invalidate refresh tokens**

SECURITY BEST PRACTICES (MUST KNOW)

- ✓ Short JWT expiry (5–15 min)
 - ✓ Refresh token rotation
 - ✓ Store refresh tokens in DB
 - ✓ HttpOnly cookies (web)
 - ✓ HTTPS only
 - ✓ Logout revokes tokens
-

COMMON MISTAKES (AVOID)

- ✗ Long-lived JWT without refresh token
 - ✗ Refresh token in localStorage
 - ✗ Not rotating refresh tokens
 - ✗ No logout invalidation
 - ✗ Trusting refresh token forever
-

INTERVIEW-PERFECT ANSWER

“We use short-lived JWTs with long-lived refresh tokens. When the access token expires, the client sends a refresh token to obtain a new JWT without re-authentication. Refresh tokens are stored securely and rotated on use.”

API Design Structures Explanation

Performance optimization is what turns a *working API* into a **fast, smooth, scalable, production-grade API**.

layers, simple language, real-world analogy, and practical code patterns.

Performance Optimization in APIs (Practical Guide)

Golden Rule (Memorize This)

Most API slowness comes from I/O

(DB, network, serialization, external calls)

So we optimize **around I/O**, not for loops.

Real-World Analogy (Simple)

Supermarket Checkout

Problem	API Equivalent
One cashier	Blocking calls
Repeated price checks	No caching
Long receipt	Large payload
Slow database	No indexing

Fixing these makes checkout fast → same with APIs.

Performance Optimization by LAYER

Client



Network



Middleware



Controller



Service



Database

We optimize **each layer**, but focus on **Service & DB**.

1 NETWORK & HTTP LEVEL (Very High Impact)

✓ Use Compression

```
builder.Services.AddResponseCompression();
```

```
app.UseResponseCompression();
```

✓ Reduces payload size

✓ Faster responses

✓ Use Correct HTTP Status & Methods

- GET → Cacheable
 - POST → Not cacheable
-

✓ Enable HTTP Caching (GET APIs)

```
[HttpGet]
```

```
[ResponseCache(Duration = 60)]
```

```
public IActionResult GetItems()
```

```
{  
    return Ok(data);  
}
```

2 MIDDLEWARE LEVEL

✗ Avoid Heavy Middleware

Bad:

- Too much logging
- Synchronous middleware
- Unnecessary exception wrapping

Good:

```
app.UseExceptionHandler();
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

API Design Structures Explanation

✅ Order Matters

```
app.UseResponseCompression();
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

Wrong order = slower pipeline.

3 CONTROLLER LEVEL (Keep It Thin)

❌ Bad Controller

```
var data = _context.Items.ToList(); // DB call
```

✅ Good Controller

```
return Ok(await _service.GetAsync());
```

Controllers should do **zero heavy work**.

4 SERVICE LEVEL (MOST IMPORTANT)

✅ Use Async Everywhere

```
public async Task<List<ItemDto>> GetAsync()
```

```
{
```

```
    return await _context.Items.ToListAsync();
```

```
}
```

❌ Never block threads:

```
_context.Items.ToList(); // BAD
```

✅ Avoid N+1 Queries

❌ BAD

```
foreach (var item in items)
```

```
{
```

```
    item.Category = _context.Categories.Find(item.CategoryId);
```

```
}
```

✅ GOOD

```
_context.Items.Include(x => x.Category).ToListAsync();
```

Select Only Required Fields

 BAD

```
_context.Items.ToListAsync();
```

 GOOD

```
_context.Items  
    .Select(x => new ItemDto(x.Id, x.Name))  
    .ToListAsync();
```

✓ Smaller memory

✓ Faster serialization

Use Pagination (Must)

```
_context.Items  
    .Skip(page * size)  
    .Take(size)  
    .ToListAsync();
```

Never return **thousands of rows**.

DATABASE LEVEL (HIGHEST IMPACT)

Index Important Columns

```
CREATE INDEX IX_Items_Name ON Items(Name);
```

Index:

- WHERE
 - JOIN
 - ORDER BY columns
-

Use AsNoTracking for Read-Only

```
_context.Items  
    .AsNoTracking()  
    .ToListAsync();
```

API Design Structures Explanation

- ✓ Faster
 - ✓ Less memory
-

Avoid Chatty DB Calls

Bad:

SaveChanges() called many times

Good:

SaveChanges() once

CACHING (GAME CHANGER)

When to Cache?

- Read-heavy data
 - Rarely changing data
-

In-Memory Cache (Simple)

```
if (!_cache.TryGetValue("items", out data))
{
    data = await _context.Items.ToListAsync();
    _cache.Set("items", data, TimeSpan.FromMinutes(5));
}
```

Distributed Cache (Redis)

- ✓ Scales across servers
 - ✓ Production standard
-

AUTH & SECURITY PERFORMANCE

Don't Query DB on Every Request for User

Use **JWT claims**, not DB calls.

API Design Structures Explanation

✓ Short JWT Payload

✗ Big token = slow

✓ Keep only userId, role

8 SERIALIZATION & RESPONSE

✓ Use DTOs (Already doing 👍)

✓ Smaller JSON

✓ Faster serialization

✓ Avoid Circular References

ReferenceHandler.IgnoreCycles

9 BACKGROUND WORK (Huge Optimization)

✗ BAD

Sending email inside request

✓ GOOD

Use background job / queue

BackgroundService.Enqueue(...)

🔥 TOP 10 PERFORMANCE RULES (SAVE THIS)

1 Async everywhere

2 Select only needed fields

3 Pagination always

4 Cache read-heavy data

5 Index DB properly

6 Avoid N+1 queries

7 Thin controllers

8 Use AsNoTracking

9 Compress responses

10 Don't block threads

API Design Structures Explanation

Interview-Perfect Answer

“API performance optimization focuses on reducing I/O cost. We use async calls, proper indexing, caching, pagination, minimal DTOs, and efficient middleware ordering to ensure fast and scalable APIs.”

Observability is what separates “API works on my machine” from “API runs safely in production”.

Explain **Logging, Metrics, and Tracing** in the **same simple, real-world way**, step by step, with **where they belong, why, and small examples**.

What is Observability? (Very Simple)

Observability = ability to understand what your API is doing from the outside

You should be able to answer:

-  Why is API slow?
-  Why did this request fail?
-  Which endpoint is most used?
-  Where is the bottleneck?

Observability answers these **without debugging production code**.

The 3 Pillars of Observability

Logs → What happened?

Metrics → How often / how slow?

Tracing → Where did time go?

All **three together** give the full picture.

Real-World Analogy (Best Way to Remember)

Factory

Factory	API
CCTV recording	Logs
Production counters	Metrics
Assembly line tracking	Tracing

API Design Structures Explanation

1 LOGGING (Most Important & Most Used)

What is Logging?

Recording important events during request processing

Examples:

- Request started
- User logged in
- Error occurred
- External API failed

What Logging is NOT

-  Console.WriteLine
-  Debugging statements
-  Logging everything

What SHOULD be logged

- ✓ Errors
- ✓ Warnings
- ✓ Important business events
- ✓ Request lifecycle info

Where Logging Belongs

Layer	Logging
Middleware	Request start/end, exceptions
Service	Business events
Controller	Minimal / none
Database	Only critical failures

Example: Logging in Middleware

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<LoggingMiddleware> _logger;
```

API Design Structures Explanation

```
public LoggingMiddleware(RequestDelegate next, ILogger<LoggingMiddleware> logger)
{
    _next = next;
    _logger = logger;
}

public async Task Invoke(HttpContext context)
{
    _logger.LogInformation(
        "Request {Method} {Path}",
        context.Request.Method,
        context.Request.Path
    );

    await _next(context);

    _logger.LogInformation(
        "Response {StatusCode}",
        context.Response.StatusCode
    );
}
```

 This gives:

- What endpoint was called
- Whether it succeeded

Example: Logging Errors in Service

```
try
{
    // business logic
```

API Design Structures Explanation

```
}  
catch (Exception ex)  
{  
    _logger.LogError(ex, "Failed to create item");  
    throw;  
}
```

Logging Levels (Memorize)

Level	When to Use
Information	Normal flow
Warning	Something odd
Error	Failure
Critical	System down

2 METRICS (Health & Performance)

What are Metrics?

Numbers over time

Examples:

- Requests per second
 - Average response time
 - Error rate
 - CPU / memory usage
-

Analogy

Speedometer in a car

- You don't read logs while driving
 - You glance at numbers
-

API Design Structures Explanation

Where Metrics Belong

- ✓ Middleware
 - ✓ Hosting layer
 - ✓ Infrastructure
 - ✗ Not inside business logic
-

Common API Metrics

Metric	Meaning
Request count	API usage
Latency	Speed
Error rate	Stability
Throughput	Load handling

Example: Measure Request Duration

```
var stopwatch = Stopwatch.StartNew();  
await _next(context);  
stopwatch.Stop();
```

```
_logger.LogInformation(  
    "Request took {Time} ms",  
    stopwatch.ElapsedMilliseconds  
);
```

 In real apps, metrics go to:

- Prometheus
 - Azure Monitor
 - CloudWatch
-

Why Metrics Matter

Without metrics:

- You don't know API is slow
 - You react after users complain
-

API Design Structures Explanation

With metrics:

- You see problems early
 - You scale safely
-

TRACING (Most Advanced, Very Powerful)

What is Tracing?

Tracking one request across the entire system

Especially important for:

- Microservices
 - External APIs
 - Background jobs
-

Analogy

 Courier tracking:

Order → Warehouse → Truck → City → Home

API tracing:

Controller → Service → DB → External API

Where Tracing Belongs

- ✓ Middleware
 - ✓ HTTP clients
 - ✓ Database calls
-

Example: Trace ID (Simple)

```
var traceId = Activity.Current?.Id ?? context.TraceIdentifier;
```

```
_logger.LogInformation("TraceId: {TraceId}", traceId);
```

 Same TraceId appears in:

- Logs
- Metrics
- Errors

This lets you:

“Follow ONE request everywhere”

API Design Structures Explanation

Distributed Tracing (Real Companies)

Tools:

- OpenTelemetry
- Jaeger
- Zipkin
- Application Insights

They show:

- Which part is slow
 - Where request failed
-

FULL OBSERVABILITY FLOW (Together)

Request arrives

↓

Logging: request start

↓

Metrics: timer starts

↓

Tracing: trace id created

↓

Service / DB / External calls

↓

Metrics: timer stops

↓

Logging: response + status

↓

Tracing: request completed

Common Observability Mistakes

-  Logging sensitive data
 -  Logging too much (noise)
 -  No correlation ID
-

API Design Structures Explanation

- ✗ No metrics for latency
- ✗ Relying only on logs

What to Use at Beginner vs Production

Level	Tools
Beginner	ILogger
Intermediate	Serilog + structured logs
Advanced	OpenTelemetry + Metrics + Tracing

Interview-Perfect Answer

“Observability consists of logging, metrics, and tracing. Logs tell us what happened, metrics show system health and performance over time, and tracing helps track a single request across services. Together, they help us monitor, debug, and scale APIs in production.”

API Design Structures Explanation

Scaling APIs is a **must-know production topic** and very commonly asked in **system design + backend interviews**.

Explain this in **very simple language**, **real-world analogy**, **clear diagrams**, and **what developers actually do**.

What is Scaling?

Scaling = ability of your API to handle more users without breaking or slowing down

Real-World Analogy (Very Easy)

Restaurant Example

Problem	Solution
More customers	Bigger kitchen OR more branches

That's it.

Vertical Scaling (Scale UP)

What is Vertical Scaling?

Increase power of one server

Example:

- 4GB RAM → 16GB RAM
 - 2 CPU → 8 CPU
-

Diagram

Users



[Bigger Server]

Pros

- ✓ Easy
 - ✓ No code change
 - ✓ Fast to apply
-

API Design Structures Explanation

✗ Cons

- ✗ Hardware limit
 - ✗ Expensive
 - ✗ Single point of failure
-

💡 When Vertical Scaling Works Best

- ✓ Small / medium apps
 - ✓ Early stage projects
 - ✓ Monolithic APIs
-

! Developer Impact

Mostly **infra-side**, not much coding.

But you **must write efficient code**, or bigger server won't help.

➡ Horizontal Scaling (Scale OUT) ★★ ★

🧠 What is Horizontal Scaling?

Add more API instances

Users



Load Balancer



API-1 API-2 API-3

📦 Core Requirement

API must be STATELESS

✗ BAD

- In-memory user sessions
 - Local cache only
 - File storage on server
-

API Design Structures Explanation

GOOD

- JWT
 - Distributed cache (Redis)
 - Cloud storage
-

Pros

- ✓ Highly scalable
 - ✓ Fault tolerant
 - ✓ Industry standard
-

Cons

-  More complexity
 -  Needs proper design
-

Stateless API (VERY IMPORTANT)

What does Stateless Mean?

Any request can go to any server

Example:

- Login → API-1
- Get profile → API-3
- Update → API-2

And it still works.

How We Achieve Statelessness

Problem	Solution
User login	JWT
Cache	Redis
Files	Blob storage
Sessions	Avoid

API Design Structures Explanation

Load Balancer Role

Distributes requests evenly

Types:

- Round-robin
- Least connections

Developer doesn't control it, but must **design API for it**.

Scaling at Different Layers

Client



Load Balancer



API Instances (Horizontal)



Service Layer



Database (Vertical + Read replicas)

Database Scaling (Often Bottleneck)

Vertical DB Scaling

- Bigger DB server

Horizontal DB Scaling

- Read replicas
 - Sharding (advanced)
-

Read Replica Example

Writes → Main DB

Reads → Replica DBs

API decides:

- GET → Read DB
 - POST/PUT → Main DB
-

API Design Structures Explanation

 **Caching = Scaling Booster**

Cache reduces DB load

Cache Flow

API

↓

Cache (Redis)

↓ (miss)

Database

Types of Cache

Type	Scope
In-memory	Single instance
Distributed	All instances

For horizontal scaling → **Distributed cache only.**

Background Processing & Scaling

 Heavy work in request

✓ Background jobs

This:

- Frees API threads
 - Improves throughput
-

Auto-Scaling (Cloud Concept)

Add/remove servers automatically

Triggers:

- CPU
 - Memory
 - Requests per second
-

API Design Structures Explanation

Your job:

- ✓ Write stateless APIs
 - ✓ Avoid shared memory
-

COMMON SCALING MISTAKES

- ✗ In-memory session state
 - ✗ File uploads stored locally
 - ✗ Large synchronous operations
 - ✗ DB calls per request
 - ✗ No caching
-

Interview-Perfect Answer

“Vertical scaling increases the capacity of a single server, while horizontal scaling adds more server instances. Modern APIs prefer horizontal scaling because it offers better fault tolerance and scalability. To support it, APIs must be stateless, use JWT for authentication, distributed caching, and externalized storage.”

Quick Decision Guide

Scenario	Scaling
Small internal app	Vertical
Public API	Horizontal
Unpredictable traffic	Horizontal
Cost-sensitive early stage	Vertical
High availability	Horizontal

API Design Structures Explanation

Rate limiting & throttling is a **must-have production feature** for any public or scalable API.

Explain it in **simple language**, **real-world analogy**, **clear rules**, and **how it fits into .NET APIs** (conceptually and practically).

Rate Limiting & Throttling (Production Guide)

One-Line Definition

Rate limiting controls how many requests a client can make in a given time window.
Throttling slows down or blocks requests when limits are exceeded.

Real-World Analogy (Very Easy)

ATM Example

ATM Rule	API Equivalent
5 withdrawals/day	Rate limit
Temporary block after many tries	Throttling
Slower response when busy	Soft throttling

Why APIs Need Rate Limiting

- ✓ Prevent abuse
- ✓ Protect backend & DB
- ✓ Ensure fair usage
- ✓ Stop brute-force attacks
- ✓ Improve stability

Without it:

One bad client can crash your API.

Rate Limiting vs Throttling

Feature	Rate Limiting	Throttling
What	Max requests	Slow/block excess
Behavior	Hard stop	Gradual
Response	429 Too Many Requests	429 / delayed

API Design Structures Explanation

Where It Belongs (Important)

Client



Rate Limiter (Middleware)



Authentication



Controller

- ✓ Before controller
 - ✓ Before DB
 - ✓ Before business logic
-

Types of Rate Limiting (Must Know)

1 Fixed Window

100 requests per minute

✗ Problem: burst at window edge

2 Sliding Window ★ (Better)

Smoothly tracks last X seconds

- ✓ More accurate
 - ✓ Less bursty
-

3 Token Bucket ★★ ★ (Most Used)

Tokens refill over time

Example:

- 10 tokens/sec
 - Each request uses 1 token
- ✓ Allows small bursts
 - ✓ Very flexible
-

API Design Structures Explanation

Leaky Bucket

Processes requests at constant rate

What Can Be Rate Limited?

Target	Example
IP Address	Public APIs
User ID	Authenticated users
API Key	Partners
Endpoint	Login / OTP

Different Limits Per Endpoint

Endpoint	Limit
Login	5/min
Search	100/min
Public GET	1000/min

Example: .NET Built-in Rate Limiter (.NET 7+)

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", config =>
    {
        config.PermitLimit = 100;
        config.Window = TimeSpan.FromMinutes(1);
        config.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
        config.QueueLimit = 0;
    });
});
app.UseRateLimiter();
[EnableRateLimiting("fixed")]
[HttpGet]
```

API Design Structures Explanation

```
public IActionResult Get()
{
    return Ok("Allowed");
}
```

Response When Limit Exceeded

HTTP 429 Too Many Requests

Retry-After: 60

Clients must retry later.

Rate Limiting in Scaled Systems

✗ In-memory limit → breaks with multiple servers

✓ Distributed store (Redis)

Distributed Rate Limiting Flow

Client

↓

API Instance

↓

Redis (counter/token)

All instances share the same limit.

Throttling Strategies

Soft Throttling

- Delay response
- Lower priority

Hard Throttling

- Reject request (429)
-

API Design Structures Explanation

Best Practices (SAVE THIS)

- 1 Apply rate limiting at **gateway / middleware**
 - 2 Different limits for different endpoints
 - 3 Stricter limits for auth endpoints
 - 4 Use distributed cache for scale
 - 5 Return clear 429 responses
 - 6 Log throttled requests
-

Common Mistakes

- ✗ Same limit for all APIs
 - ✗ No limit on login
 - ✗ In-memory limit with load balancer
 - ✗ Silent throttling (no feedback)
-

Interview-Perfect Answer

“Rate limiting controls how many requests a client can make within a time window, while throttling handles excess requests by delaying or rejecting them. They are typically implemented as middleware using strategies like token bucket or sliding window and are essential for protecting APIs, ensuring fairness, and maintaining performance.”

When to Use What?

Scenario	Strategy
Login / OTP	Fixed window
Public API	Token bucket
Search APIs	Sliding window
High traffic	Distributed rate limiting

API Gateway deep dive

API Gateway is the **front door** of modern backend systems.

If you understand this well, you're thinking like a **senior backend / system designer**.

API Gateway — Deep Dive (Industry Standard)

One-Line Definition (MEMORIZE)

API Gateway is a **single entry point** that sits in front of one or many backend services and handles cross-cutting concerns centrally.

Real-World Analogy (Best Way to Understand)

Corporate Office Reception

Office	API World
Reception desk	API Gateway
Security check	Authentication
Visitor rules	Rate limiting
Directing visitors	Routing
Visitor logs	Logging
Blocking troublemakers	Throttling

 Visitors never talk directly to internal teams.

Why API Gateway Exists (The Core Problem)

Without API Gateway:

Client → Auth Service

Client → Order Service

Client → Payment Service

Client → Inventory Service

-  Duplicate logic
 -  Security everywhere
 -  Hard to manage
 -  Clients tightly coupled
-

With API Gateway:

Client

↓

API Design Structures Explanation

API Gateway



Auth | Order | Payment | Inventory

- ✓ One entry point
 - ✓ Central security
 - ✓ Easier scaling
 - ✓ Cleaner services
-

What Problems API Gateway Solves

1 Authentication & Authorization (MOST COMMON)

- ✓ Validate JWT
- ✓ Validate API keys
- ✓ Check roles/claims

Backend services trust the gateway

2 Rate Limiting & Throttling

- ✓ Per user
- ✓ Per IP
- ✓ Per API

Protects **entire system**, not just one service.

3 Routing & Load Balancing

/api/orders → Order Service

/api/users → User Service

/api/payments → Payment Service

Gateway decides **where to send requests**.

4 Request / Response Transformation

- ✓ Change headers
- ✓ Change payload
- ✓ Version mapping

Example:

- Client sends v1
 - Backend runs v2
-

API Design Structures Explanation

5 Caching (Massive Performance Boost)

- ✓ Cache GET responses
 - ✓ Reduce DB load
 - ✓ Faster response
-

6 Logging, Metrics, Tracing (Observability)

- ✓ Centralized logs
 - ✓ Centralized metrics
 - ✓ Request correlation
-

7 Security Shield

- ✓ Hide internal services
 - ✓ Block malicious traffic
 - ✓ IP whitelisting
-

🧠 What API Gateway is NOT

- ✗ Business logic
- ✗ Database access
- ✗ Complex computation

👉 Gateway = traffic controller, not worker

🏠 API Gateway Architecture (Common)

Client

↓

API Gateway

↓

Service A Service B Service C

↓

↓

Database Database

Services:

- Are small
 - Stateless
 - Focus on business logic
-

API Design Structures Explanation

API Gateway vs Reverse Proxy

Feature	Reverse Proxy	API Gateway
Routing	✓	✓
Load balancing	✓	✓
Authentication	✗	✓
Rate limiting	✗	✓
Request transformation	✗	✓
Observability	✗	✓

 API Gateway = Smart Reverse Proxy

Where API Gateway Fits in Scaling

Users

↓

API Gateway (scaled)

↓

Microservices (scaled)

↓

Databases

Gateway itself is:

- ✓ Stateless
- ✓ Horizontally scalable

Authentication Flow with Gateway

Client → Gateway

- Validate JWT
- Extract claims

↓

Backend Service

- Trust gateway

Backend:

- ✓ No auth duplication
- ✓ Cleaner code

API Design Structures Explanation

Rate Limiting Flow

Client



Gateway

- Count requests
- Apply limits



Service

If blocked:

HTTP 429 Too Many Requests

API Versioning with Gateway

/api/v1/orders → OrderService v1

/api/v2/orders → OrderService v2

OR

Client v1 → Gateway → Backend v2

Gateway maps versions.

Popular API Gateway Options (FYI)

Type	Examples
Cloud	Azure API Management, AWS API Gateway
Open source	Kong, NGINX, Traefik
Self-hosted (.NET)	Ocelot, YARP

 **Concept > tool** (interviews focus on concepts)

When SHOULD You Use API Gateway?

- ✓ Microservices
 - ✓ Public APIs
 - ✓ Multiple clients (web, mobile)
 - ✓ Need rate limiting & security
-

API Design Structures Explanation

When You DON'T Need It

-  Small monolith
-  Internal-only APIs
-  Very simple apps

Don't over-engineer.

Common API Gateway Mistakes

-  Business logic in gateway
 -  Overloading gateway
 -  Single gateway for everything (no scaling)
 -  No fallback / resilience
-

Interview-Perfect Answer (MEMORIZE)

“An API Gateway acts as a single entry point to backend services and handles cross-cutting concerns such as authentication, authorization, rate limiting, routing, caching, and observability. It simplifies clients, centralizes security, and enables scalable microservice architectures.”

Simple Mental Model (FINAL)

API Gateway = Traffic Police

Microservices = Workers

API Design Structures Explanation

Payment Gateway architecture is a **classic real-world system design problem** and very common in interviews and real projects.

Explain it **step-by-step**, **very simple**, **no finance jargon**, and **API-first**, so you can use this design in **any application**.

Payment Gateway Architecture (Simple & Production-Ready)

One-Line Idea (MEMORIZE)

Your system never handles card details directly.

It talks to a Payment Gateway via APIs and reacts to success/failure securely.

Real-World Analogy (Best Way to Understand)

Shop & Bank

Real World	Software
Customer	User
Shop	Your Backend API
Bank	Payment Gateway
Receipt	Payment Status

 The shop **never stores card details**.

It only asks the bank: *"Was payment successful?"*

High-Level Architecture

Client (Web / Mobile)



Your Backend API



Payment Gateway API



Bank / Network

Your backend is the **middleman**, not the card processor.

CORE SECURITY RULE (VERY IMPORTANT)

 **NEVER store:**

- Card number
 - CVV
 - Expiry
-

API Design Structures Explanation

- ✓ Let **payment gateway** handle it
 - ✓ Your API handles **order & payment status only**
-

FULL PAYMENT FLOW (STEP-BY-STEP)

We'll break this into **clear stages**.

1 User Starts Payment

User clicks **Pay Now**

Client → POST /api/payments/create

Request:

```
{  
  "orderId": "ORD123",  
  "amount": 1500  
}
```

2 Backend Creates Payment Intent (NOT Payment)

 This step means:

"I plan to take money, but haven't taken it yet"

Backend:

- Validates order
- Stores payment record as Pending

Payment table:

- PaymentId
 - OrderId
 - Amount
 - Status = Pending
-

3 Backend Calls Payment Gateway API

Backend sends request to gateway:

Backend → Payment Gateway

Payload:

```
{  
  "amount": 1500,  
}
```

API Design Structures Explanation

```
"currency": "INR",  
"referenceId": "ORD123",  
"callbackUrl": "/api/payments/webhook"  
}
```

Gateway responds with:

```
{  
  "paymentSessionId": "ps_abc123",  
  "redirectUrl": "https://gateway.com/pay"  
}
```

4 Client Redirects to Payment Gateway

Client → redirectUrl

User enters:

- Card / UPI / NetBanking

✓ This happens **outside your system**

✓ You never see card data

5 User Completes Payment

Gateway processes:

- Bank authorization
- Fraud checks
- OTP / 3DS

Outcome:

✓ Success

✗ Failure

6 Gateway Calls Your Webhook (MOST IMPORTANT)

🧠 Webhook = Gateway → Your API (server to server)

Gateway → POST /api/payments/webhook

Payload:

```
{  
  "paymentSessionId": "ps_abc123",  
  "status": "SUCCESS",  
}
```

API Design Structures Explanation

```
"transactionId": "TXN789"
}
```

Backend Verifies & Updates DB

Backend must:

-  Verify signature (security)
-  Match paymentSessionId
-  Update payment status

Payment Status:

- SUCCESS / FAILED

 **Never trust client response**

 Trust webhook only

Backend Updates Order Status

If payment SUCCESS:

Order → Confirmed

Else:

Order → PaymentFailed

Client Checks Final Status

Client polls or calls:

GET /api/payments/status?orderId=ORD123

Response:

```
{
  "status": "SUCCESS"
}
```

COMPLETE FLOW (ONE VIEW)

Client

↓ (Create Payment)

Backend API

↓ (Create intent)

Payment Gateway

↓ (Redirect)

API Design Structures Explanation

User pays



Payment Gateway



Backend API



Database



Client gets result



Why Webhook is CRITICAL

- ✗ Client can lie
- ✗ Network may fail
- ✗ Redirect may be interrupted

✓ Webhook is:

- Secure
- Reliable
- Server-to-server



Final payment status always comes from webhook



Backend Folder Responsibilities (Payment)

Payments

- └─ PaymentsController
- └─ PaymentService
- └─ PaymentGatewayClient
- └─ WebhookController
- └─ DTOs
- └─ Entities

Responsibilities:

- Controller → HTTP
- Service → Business logic
- GatewayClient → External API
- Webhook → Verification

API Design Structures Explanation

- DTO → Data shape
- Entity → DB

Failure Scenarios (Real World)

Case	Handling
User closes browser	Webhook still updates
Payment slow	Pending → Success later
Duplicate webhook	Idempotency
Gateway down	Retry / fallback

Idempotency (VERY IMPORTANT)

Same webhook may come multiple times

Solution:

- Check if payment already processed
- Ignore duplicates

Best Practices (SAVE THIS)

- 1 Always use webhooks
- 2 Never trust client status
- 3 Keep payment state machine
- 4 Log all payment events
- 5 Make webhook idempotent
- 6 Separate payment & order logic

Interview-Perfect Answer

“A payment gateway integration involves creating a payment intent on the backend, redirecting the client to the gateway for payment, and receiving the final payment status through secure webhooks. The backend never handles card data directly and updates order state only after verifying the webhook.”

FINAL MENTAL MODEL (REMEMBER)

Your API = Coordinator

Gateway = Money Handler

Webhook = Source of Truth

API Design Structures Explanation

These two concepts (**Webhooks + Saga pattern**) are the **heart of real payment systems**.
I'll explain them **very simply, step-by-step**, with **real-world examples**, so you can **visualize** instead of memorizing.

PART 1: Webhooks (Simple & Clear)

One-Line Definition (MEMORIZE)

A webhook is when one system calls another system automatically when something happens.

Not you asking.
They notify you.

Real-World Analogy (Best One)

Courier Delivery

Real World	Webhook
Courier calls you	Gateway calls your API
"Package delivered"	"Payment successful"
You didn't ask	It notified you

 That's a webhook.

Normal API Call (You Ask)

Your App → Payment Gateway

"Did payment succeed?"

Problems:

- Slow
 - Can miss updates
 - Not reliable
-

Webhook (They Tell You)

Payment Gateway → Your API

"Payment SUCCESS"

Reliable. Secure. Automatic.

Webhook Payment Flow (Step-by-Step)

You Register a Webhook URL

API Design Structures Explanation

When setting up gateway:

`https://yourapi.com/api/payments/webhook`

2 Payment Happens

User completes payment on gateway page.

3 Gateway Calls Your Webhook

POST `/api/payments/webhook`

Payload:

```
{  
  "orderId": "ORD123",  
  "paymentId": "PAY789",  
  "status": "SUCCESS",  
  "amount": 1500  
}
```

4 Your API Verifies Request

VERY IMPORTANT:

- ✓ Verify signature
 - ✓ Check paymentId exists
 - ✓ Prevent duplicates
-

5 Update Database

Payment → SUCCESS

Order → CONFIRMED

Why Webhooks are CRITICAL in Payments

Problem	Webhook Solves
User closes browser	✓
Network failure	✓
Redirect breaks	✓
Client lies	✓

 Webhook is the SOURCE OF TRUTH

API Design Structures Explanation

Common Webhook Mistakes

-  Trust client response
 -  No signature validation
 -  No idempotency
 -  Slow webhook response
-

Simple Rule

Webhook must be fast, idempotent, and secure

PART 2: Saga Pattern for Payments (VERY IMPORTANT)

Payments are **not one step**.

They involve:

- Order
- Payment
- Inventory
- Notification

Saga manages this **safely**.

One-Line Definition (MEMORIZE)

Saga is a way to manage multiple steps across services using events and compensations instead of transactions.

Real-World Analogy

Movie Ticket Booking

-  1 Book seat
-  2 Pay money
-  3 Send ticket

If payment fails:

-  Cancel seat

That's a saga.

Why Normal Transactions Fail

You **cannot** do:

Order DB + Payment DB + Inventory DB

API Design Structures Explanation

in one transaction.

So we use **Saga**.

Payment Saga (Step-by-Step)

Services Involved

Order Service

Payment Service

Inventory Service

Notification Service

Step 1: Order Created

Order Service:

Status = PENDING

Event:

OrderCreated

Step 2: Payment Service

Receives event:

OrderCreated

Tries payment:

- Calls payment gateway
 - Waits for webhook
-

Step 3: Payment Result

✓ SUCCESS

Event:

PaymentSuccessful

✗ FAILURE

Event:

PaymentFailed

API Design Structures Explanation

Step 4: Saga Reactions

If SUCCESS

Service	Action
Inventory	Reserve items
Order	Confirm
Notification	Send email

If FAILURE (Compensation)

Service	Compensation
Order	Cancel
Inventory	Release stock
Notification	Payment failed

Visual Saga Flow

OrderCreated

↓

PaymentProcessing

↓

PaymentSuccess → Confirm Order

OR

PaymentFailed → Cancel Order

Compensation ≠ Rollback

✗ DB rollback

✓ Logical undo action

Example:

- Undo payment → Refund
 - Undo reservation → Release stock
-

Types of Saga

1 Choreography (Event-Based) ★★ ★

- No central controller
-

API Design Structures Explanation

- Services react to events

✓ Simple

✓ Scalable

✗ Hard to debug

2 Orchestration ★★ ★

- Central Saga controller
- Controls steps

✓ Clear flow

✓ Easier error handling

✗ Slight coupling

🧠 Which One for Payments?

👉 Orchestration preferred

Payments need:

- Clear state
 - Auditing
 - Retry logic
-

📦 Saga State Example

SagaId

OrderId

CurrentStep

Status

🔥 Best Practices (SAVE THIS)

- 1 Webhooks = source of truth
 - 2 Saga handles multi-service flow
 - 3 Never use distributed transactions
 - 4 Always have compensation logic
 - 5 Use events for communication
 - 6 Make every step idempotent
-

API Design Structures Explanation

Interview-Perfect Answer

“Webhooks are server-to-server callbacks used to reliably receive payment status from a gateway. The Saga pattern coordinates multiple services involved in payment processing using events and compensation steps instead of distributed transactions, ensuring consistency across the system.”

FINAL MENTAL MODEL (REMEMBER)

Webhook = Notification

Saga = Coordinator

Event-Driven Architecture (EDA) Deep Dive

Event-Driven Architecture (EDA) is the backbone of **scalable systems, payments, microservices, real-time platforms, and modern cloud applications.**

I'll go **deep**, but keep it **simple, step-by-step**, with **real-world examples, payment context, dos & don'ts**, and **interview-ready explanations.**

⚡ Event-Driven Architecture (EDA) — Deep Dive

🧠 One-Line Definition (MEMORIZE)

Event-Driven Architecture is a design where services communicate by **publishing and reacting to events** instead of calling each other directly.

🏠 Real-World Analogy (Best One)

📢 Announcement System in a Mall

Real World	EDA
Announcement: "Store closed"	Event published
All shops hear it	Subscribers
Each shop reacts differently	Independent services

No one calls anyone directly.

❌ Traditional (Request-Response) Architecture

Order Service → Payment Service

Order Service → Inventory Service

Order Service → Notification Service

Problems:

- ❌ Tight coupling
- ❌ Cascading failures
- ❌ Slow
- ❌ Hard to scale

✅ Event-Driven Architecture

Order Service

↓

OrderCreated Event

↓

API Design Structures Explanation

Event Broker

↓ ↓ ↓

Payment Inventory Notification

- ✓ Loose coupling
 - ✓ Scalable
 - ✓ Resilient
 - ✓ Async
-

Core Building Blocks of EDA

1 Event

Something that already happened

Examples:

- OrderCreated
- PaymentCompleted
- UserRegistered

Events are:

- ✓ Past-tense
 - ✓ Immutable
 - ✓ Facts
-

2 Producer (Publisher)

Service that emits events

Example:

- Order Service publishes OrderCreated
-

3 Consumer (Subscriber)

Service that reacts to events

Example:

- Payment Service listens to OrderCreated
-

4 Event Broker (Message Broker)

Middleman that delivers events

Examples:

- Kafka
-

API Design Structures Explanation

- RabbitMQ
- Azure Service Bus

Broker:

- ✓ Stores events
- ✓ Delivers reliably
- ✓ Handles retries

Event Flow (Simple)

Producer → Broker → Consumer

Producer **doesn't know** who consumes.

Why EDA is Powerful

Problem	Solved By
Tight coupling	Events
Sync failures	Async
Scaling services	Independent consumers
Long workflows	Event chains

Payment System Example (Real World)

Without EDA ❌

Order → Payment → Inventory → Notification

If Payment fails → whole chain breaks.

With EDA ✅

OrderCreated

↓

Broker

↓

Payment Service → PaymentCompleted

↓

Broker

↓

Inventory, Notification

API Design Structures Explanation

- ✓ Each service independent
- ✓ Failure isolated
- ✓ Retryable

Event vs Command (VERY IMPORTANT)

Event	Command
Fact	Instruction
Past	Present
Fire-and-forget	Expect result

Example:

- Event: PaymentCompleted
- Command: ProcessPayment

EDA uses **events**, not commands.

Event-Driven Architecture Styles

1 Event Choreography ★★ ★

No central controller.

OrderCreated

↓

Payment reacts

↓

PaymentCompleted

↓

Inventory reacts

- ✓ Loose coupling
- ✓ Scalable
- ✗ Hard to trace flow

2 Event Orchestration ★★ ★

Central controller (Saga).

Saga → Order → Payment → Inventory

- ✓ Clear control
- ✓ Easier debugging
- ✗ More coordination logic

API Design Structures Explanation

 **Payments usually use Orchestration**

 **Eventual Consistency (CRITICAL CONCEPT)**

Data becomes consistent over time, not instantly

Example:

- Order shows Pending
- Payment completes
- Order becomes Confirmed

This is **normal** in EDA.

 **Idempotency (NON-NEGOTIABLE)**

Same event may be delivered multiple times

Consumer must:

- ✓ Detect duplicates
- ✓ Process only once

Example:

If EventId already processed → ignore

 **Common EDA Problems & Solutions**

 **Event Loss**

- ✓ Durable broker

 **Duplicate Events**

- ✓ Idempotency

 **Ordering Issues**

- ✓ Partitioning / sequence

 **Debugging Difficulty**

- ✓ Correlation IDs
-

 **Event Design Best Practices**

- ✓ Use meaningful names (OrderCreated)
 - ✓ Include eventId, timestamp
 - ✓ Avoid huge payloads
-

API Design Structures Explanation

- ✓ Never change old events
 - ✓ Version events carefully
-

Event Payload Example

```
{  
  "eventId": "evt_123",  
  "eventType": "PaymentCompleted",  
  "orderId": "ORD789",  
  "amount": 1500,  
  "timestamp": "2025-01-10T10:30:00Z"  
}
```

When to Use Event-Driven Architecture

- ✓ Microservices
 - ✓ Payments
 - ✓ Notifications
 - ✓ High scalability
 - ✓ Loose coupling
-

When NOT to Use It

- ✗ Simple CRUD apps
 - ✗ Small monoliths
 - ✗ Immediate consistency required
-

Interview-Perfect Answer (SAVE THIS)

“Event-Driven Architecture is a design where services communicate through events published to a broker. Producers emit events without knowing consumers, enabling loose coupling, scalability, and resilience. It is commonly used in payment systems, microservices, and asynchronous workflows.”

FINAL MENTAL MODEL (REMEMBER)

API Call = Ask

Event = Tell

Ask → request/response

Tell → event-driven

Full E-commerce system design (end-to-end)

This is **end-to-end system design** level.

I'll design a **REAL, production-ready E-commerce system** using:

- **Frontend:** Angular 20
- **Backend:** .NET 6 / 7 / 8 / 9 (compatible design)
- **Database:** SQL Server
- **Architecture:** Scalable, maintainable, company-standard

I'll explain **WHAT, WHY, and HOW** — step by step, in **simple language**.

FULL E-COMMERCE SYSTEM DESIGN (END-TO-END)

BIG PICTURE (ONE VIEW)

Frontend (Angular 20)



API Gateway



Backend APIs (.NET)



Databases (SQL)



External Services (Payment, Email)

1 HIGH-LEVEL ARCHITECTURE

Client (Web / Mobile)



API Gateway



| Auth Service |

| User Service |

| Product Service |

| Cart Service |

| Order Service |

API Design Structures Explanation

| Payment Service |
| Inventory Service |
| Notification Service |

↓

SQL Databases (per service)

- ✓ Microservice-ready
- ✓ Can start as **modular monolith**
- ✓ Easily scalable later

2 FRONTEND (ANGULAR 20)

Frontend Responsibilities

- ✓ UI / UX
- ✓ Form validation
- ✓ API calls
- ✓ State management
- ✓ JWT storage
- ✗ No business logic
- ✗ No payment logic

Angular 20 Folder Structure

src/

```
└─ app/  
  └─ core/  
    └─ services/  
    └─ guards/  
    └─ interceptors/  
    └─ models/  
  └─ features/  
    └─ auth/  
    └─ products/  
    └─ cart/  
    └─ checkout/  
    └─ orders/
```


API Design Structures Explanation

```
|  └─ shared/  
|  └─ app.routes.ts
```

Frontend Auth Flow

- 1 Login → get JWT
 - 2 Store token (memory/localStorage)
 - 3 Attach JWT via HTTP Interceptor
 - 4 Route guards protect pages
-

Frontend Checkout Flow

Browse → Add to Cart → Checkout → Redirect to Payment → Status Page

3 BACKEND (.NET API DESIGN)

Backend Principles

- ✓ Clean Architecture
 - ✓ DTOs
 - ✓ Async everywhere
 - ✓ Stateless APIs
 - ✓ JWT Authentication
-

Backend Folder Structure (Standard)

src/

```
└─ API/  
|  └─ Controllers/  
|  └─ Middlewares/  
|  └─ Program.cs  
└─ Application/  
|  └─ DTOs/  
|  └─ Interfaces/  
|  └─ Services/  
|  └─ Validators/  
└─ Domain/  
|  └─ Entities/  
|  └─ Enums/
```

API Design Structures Explanation

└─ Infrastructure/
| └─ DbContext/
| └─ Repositories/
| └─ ExternalServices/

4 AUTHENTICATION & AUTHORIZATION

Auth Flow

Login → JWT Access Token

→ Refresh Token

JWT contains:

- UserId
 - Role
-

API Security

- ✓ JWT validation middleware
 - ✓ Role-based authorization
 - ✓ Rate limiting
 - ✓ API Gateway protection
-

5 PRODUCT SERVICE

Responsibilities

- ✓ Products
 - ✓ Categories
 - ✓ Pricing
-

SQL Tables

Products

Categories

ProductImages

API Example

GET /api/products

GET /api/products/{id}

API Design Structures Explanation

6 CART SERVICE

Responsibilities

- ✓ Add/remove items
 - ✓ Calculate totals
 - ✓ Temporary storage
-

Design Choice

- ✓ Cart stored in DB (logged-in users)
 - ✓ CartId per user
-

Tables

Carts

CartItems

7 ORDER SERVICE

Responsibilities

- ✓ Create order
 - ✓ Track order status
 - ✓ Order history
-

Order State Machine

CREATED

→ PAYMENT_PENDING

→ PAID

→ SHIPPED

→ DELIVERED

Tables

Orders

OrderItems

API Design Structures Explanation

8 PAYMENT SERVICE (CRITICAL)

Responsibilities

- ✓ Create payment intent
 - ✓ Call payment gateway
 - ✓ Handle webhooks
 - ✓ Update order
-

Payment Flow (Simplified)

Create Order

↓

Create Payment Intent

↓

Redirect to Gateway

↓

Webhook → Payment Result

↓

Update Order

Tables

Payments

PaymentEvents

Webhook Rule

Webhook = source of truth

- ✓ Signature verification
 - ✓ Idempotency
-

9 INVENTORY SERVICE

Responsibilities

- ✓ Stock count
 - ✓ Reserve stock
 - ✓ Release stock
-

API Design Structures Explanation

Inventory Flow

Order Created → Reserve Stock

Payment Failed → Release Stock

1 0 EVENT-DRIVEN FLOW (SAGA)

Events Used

OrderCreated

PaymentSuccessful

PaymentFailed

OrderCancelled

Saga Example

OrderCreated

↓

Payment Service

↓

PaymentSuccess

↓

Inventory Reserve

↓

Order Confirmed

Compensation:

PaymentFailed → Release Inventory → Cancel Order

1 1 NOTIFICATION SERVICE

Responsibilities

✓ Email

✓ SMS

✓ Push notifications

Triggered by:

- Events only
-

API Design Structures Explanation

1 2 DATABASE DESIGN (SQL)

Pattern Used

- ✓ Database per service (logical)
 - ✓ Same SQL server allowed initially
-

Indexing Strategy

- ✓ ProductId
 - ✓ UserId
 - ✓ OrderId
-

1 3 PERFORMANCE & SCALING

Techniques Used

- ✓ Caching (Redis later)
 - ✓ Pagination
 - ✓ Async APIs
 - ✓ Background jobs
 - ✓ Stateless services
-

1 4 OBSERVABILITY

- ✓ Central logging
 - ✓ Metrics (latency, error rate)
 - ✓ Tracing (correlationId)
-

1 5 API GATEWAY ROLE

- ✓ Auth
 - ✓ Rate limiting
 - ✓ Routing
 - ✓ Logging
 - ✓ Versioning
-

1 6 SECURITY CHECKLIST

- ✓ HTTPS only
 - ✓ JWT
 - ✓ Webhook signature validation
 - ✓ Input validation
 - ✓ SQL parameterized queries
-

API Design Structures Explanation

INTERVIEW-PERFECT SUMMARY

“The e-commerce system uses Angular 20 for frontend, .NET APIs for backend, and SQL for data storage. It follows clean architecture, JWT-based authentication, event-driven workflows for orders and payments, webhook-based payment confirmation, and Saga pattern for consistency. The system is scalable, secure, and production-ready.”

FINAL MENTAL MODEL (REMEMBER)

Frontend = Experience

Backend = Rules

Events = Glue

Payments = Webhooks + Saga

DB = Ownership

Database optimization & indexing deep dive

Database optimization & indexing is where **real performance gains** happen.

A slow API is **almost always** a slow or badly used database.

We will go **deep**, but keep it **simple, practical**, and **SQL Server-focused**, exactly how companies expect.

Database Optimization & Indexing (Deep Dive)

One-Line Truth (MEMORIZE)

A good database design + correct indexes can make queries 100x faster without changing API code.

Real-World Analogy (Very Easy)

Library Example

Library	Database
No index	Searching every book
Index (catalog)	Direct shelf lookup
Wrong index	Slow search
Too many indexes	Slow book updates

Indexes = **fast lookup**, but not free.

WHERE PERFORMANCE IS LOST (Reality)

90% issues come from:

- Missing indexes
 - Wrong indexes
 - Large scans
 - Chatty queries
 - Returning unnecessary data
-

Database Optimization Has 4 Pillars

- 1 Schema Design
- 2 Indexing
- 3 Query Optimization
- 4 Operational Practices

We'll cover all.

API Design Structures Explanation

1 SCHEMA DESIGN (FOUNDATION)

✓ Use Correct Data Types

✗ BAD

VARCHAR(255) for everything

✓ GOOD

INT, BIGINT, DATETIME2, DECIMAL(18,2)

Why?

- Less storage
- Faster comparisons
- Better indexes

✓ Normalize (But Not Too Much)

- Normalize core data (Orders, Users)
- Avoid extreme joins for read-heavy paths

Use **controlled denormalization** where needed.

✓ Primary Keys (ALWAYS)

PRIMARY KEY (Id)

✓ Clustered by default

✓ Used heavily in joins

2 INDEXING (MOST IMPORTANT)

🧠 What is an Index?

A data structure that allows SQL Server to find rows quickly without scanning the entire table

🔑 Types of Indexes (MUST KNOW)

1 Clustered Index

- Defines physical row order
- Only ONE per table
- Usually on Id

PRIMARY KEY CLUSTERED (Id)

API Design Structures Explanation

2 Non-Clustered Index ★★ ★

- Separate lookup structure
- Multiple allowed

```
CREATE INDEX IX_Orders_UserId
```

```
ON Orders(UserId);
```

3 Composite Index (Multi-column)

Order matters!

```
CREATE INDEX IX_Orders_UserId_Status
```

```
ON Orders(UserId, Status);
```

Used for:

```
WHERE UserId = @id AND Status = 'PAID'
```

4 Covering Index ★★ ★

Index includes all needed columns.

```
CREATE INDEX IX_Orders_UserId
```

```
ON Orders(UserId)
```

```
INCLUDE (OrderDate, TotalAmount);
```

- ✓ No table lookup
 - ✓ Very fast
-

✗ Over-Indexing Problem

Too many indexes:

- Slows INSERT
- Slows UPDATE
- Consumes memory

Rule:

Index **read-heavy paths**, not everything.

3 WHAT SHOULD BE INDEXED?

Index columns used in:

- ✓ WHERE
 - ✓ JOIN
-

API Design Structures Explanation

✓ ORDER BY

✓ GROUP BY

Example (E-commerce)

Orders(UserId, CreatedDate)

OrderItems(OrderId)

Payments(OrderId, Status)

Products(CategoryId)

QUERY OPTIMIZATION (VERY IMPORTANT)

SELECT *

SELECT * FROM Orders;

✓ Reads unnecessary data

✓ Breaks covering indexes

Select Only Needed Columns

SELECT Id, TotalAmount FROM Orders;

Functions in WHERE

WHERE YEAR(CreatedDate) = 2025

 Index NOT used

Use Range Queries

WHERE CreatedDate >= '2025-01-01'

AND CreatedDate < '2026-01-01'

✓ Index-friendly

LIKE '%text%'

WHERE Name LIKE '%phone%'

 Full scan

Use:

- Full-text index
 - Search service
-

5 EXECUTION PLANS (SECRET WEAPON)

What is Execution Plan?

Shows **how SQL Server executes your query**

Look for:

- Index Seek 
 - Index Scan 
 - Table Scan  
-

Example

Index Seek → Good

Table Scan → Problem

6 TRANSACTIONS & LOCKING

Long Transactions

- Block other queries
 - Cause timeouts
-

Best Practices

- ✓ Keep transactions short
 - ✓ No external calls inside transaction
 - ✓ Correct isolation level
-

7 READ PERFORMANCE OPTIMIZATION

★ Use AsNoTracking (EF Core)

```
_context.Orders.AsNoTracking().ToListAsync();
```

- ✓ Faster
 - ✓ Less memory
-

★ Pagination (MANDATORY)

OFFSET 0 ROWS FETCH NEXT 20 ROWS ONLY

Never return huge datasets.

API Design Structures Explanation

8 WRITE PERFORMANCE OPTIMIZATION

- ✓ Batch inserts
 - ✓ Avoid per-row SaveChanges
 - ✓ Reduce indexes on write-heavy tables
-

9 CACHING STRATEGY (BIG BOOST)

Cache:

- Product list
- Categories
- Config data

Avoid caching:

- Payments
 - Stock counts (unless careful)
-

10 MONITORING & MAINTENANCE

Monitor

- Slow queries
 - Deadlocks
 - Index usage
-

Index Maintenance

- ✓ Rebuild fragmented indexes
 - ✓ Remove unused indexes
-

COMMON DATABASE MISTAKES

- ✗ Missing indexes
 - ✗ SELECT *
 - ✗ No pagination
 - ✗ Chatty DB calls
 - ✗ Business logic in SQL
-

API Design Structures Explanation

INTERVIEW-PERFECT ANSWER (SAVE THIS)

“Database optimization involves efficient schema design, proper indexing on frequently queried columns, optimized SQL queries, and monitoring execution plans. Indexes improve read performance but must be balanced against write cost. Techniques like pagination, covering indexes, and avoiding full table scans are essential for scalable systems.”

FINAL CHEAT SHEET (MEMORIZE)

- Index WHERE/JOIN columns
- Avoid SELECT *
- Use execution plans
- Pagination always
- AsNoTracking for reads
- Fewer, better indexes